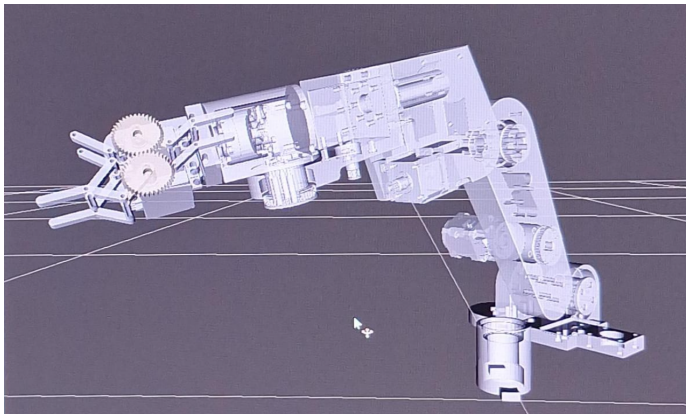
# Entwicklung eines 6dof Roboterarms



ASIT-Projekt verfasst von

**Matteo Gassler**
**Emanuel Käfer**
**Alexander Dick**

Betreuer: Prof. Dr. Gerhard Gaube

HTBLA Saalfelden

Abteilung für Elektrotechnik

Abgabevermerk:

Saalfelden, 2 April 2026 übernommen von:

# Vorwort

Vorwort Einfügen

--

Ort Datum Einfügen

Matteo Gassler & Emanuel Käfer & Alexander Dick

# Abstract-Deutsch

abstrakt deutsch einfügen

# Abstract-Englisch

abstract english einfügen

# Inhaltsverzeichnis

**Vorwort** ii

**Abstract-Deutsch** iii

**Abstract-Englisch** iv

**1 Projektüberblick** 1

1.1 Zielsetzung . . . . . 1

1.2 Umsetzung . . . . . 1

**2 Einleitung** 2

**3 Low Level Architektur und Steuerung** 3

3.1 Implementierung der 6 Achsen und Skalierbarkeit . . . . . 3

3.2 Initialisierung und SPI-Kommunikation der Motortreiber . . . . . 5

3.3 Finite State Machine (Endlicher Zustandsautomat) . . . . . 7

3.4 Kinematische Referenzierungsfahrt (Homing) . . . . . 8

3.5 Verbindung mir Raspberry Pi und normaler Betrieb . . . . . 11

**4 High Level Software** 12

4.1 Inverse Kinematik und IKpy . . . . . 12

4.2 Hypertext Markup Language-HTML . . . . . 12

4.3 FastAPI . . . . . 12

**5 Hardware** 13

**6 Elektronikdesign und Hardware-Architektur** 14

6.1 Systemarchitektur und Signalfluss . . . . . 14

6.2 Komponentenauswahl und Begründung . . . . . 14

6.3 Mikrocontroller-Infrastruktur und Peripherie . . . . . 15

6.4 Schnittstellen und Bus-Topologie . . . . . 16

6.5 Spannungsversorgung und Power-Management . . . . . 16

6.6 Prototyping und Hardware-Verifikation . . . . . 17

6.7 Schaltplan und PCB-Layout (Integriertes Design) . . . . . 18

**Quellenverzeichnis** **20**

# 1 Projektüberblick

## 1.1 Zielsetzung

Da die Entwicklung unseres 6dof Roboterarms im Rahmen des Faches ASIT erfolgt und der Lernerfolg im Vordergrund steht konnten wir uns die Aufgabenstellung selbst stellen und hatten dabei keine großen Einschränkungen. Wir haben uns dazu entschieden das der Roboterarm als Mindestanforderung eine Pick and Place Operation, also ein Objekt aufheben und wo anders platzieren können soll. Als optionale weitere Herausforderung soll man über ein Graphical user Interface Koordinaten eingeben können zu denen sich der Arm dann bewegt.

## 1.2 Umsetzung

# 2 Einleitung

# 3 Low Level Architektur und Steuerung

Dieses Kapitel beschreibt die Implementierung der zentralen Low-Level-Robotersteuerung, die auf einem Mikrocontroller läuft. Sie stellt das funktionale Bindeglied zwischen der mechanischen Konstruktion (siehe 5) und der elektrischen Leistungselektronik (siehe 6) dar. Die Firmware wurde in Bare-Metal-C für einen STM32F446-Mikrocontroller in der Entwicklungsumgebung STM32CubeIDE entwickelt. Ziel der Softwarearchitektur ist es, harte Echtzeitanforderungen für die synchrone Ansteuerung von sechs Schrittmotoren zu erfüllen und gleichzeitig eine hochgradig skalierbare Basis für zukünftige Pfadplanungen zu schaffen.

## 3.1 Implementierung der 6 Achsen und Skalierbarkeit

Bei der Ansteuerung eines 6-Achs-Roboterarms führt die Zuweisung von Mikrocontroller-Pins als Einzelvariablen und die statische Zuweisung von Peripherie-Registern schnell zu eingeschränktem und fehleranfälligem Programmcode. Um dies zu vermeiden, wurde eine Pointer-Array-basierte Datenstruktur implementiert. Anstatt jede Achse separat zu adressieren, werden Pointer auf die entsprechenden Hardware-Timer (`TIM_HandleTypeDef`), PWM-Kanäle sowie die GPIO-Ports für Richtungs- (`DIR`) und Chip-Select-Signale (`CS`) in globalen Arrays gekapselt.
Durch diese Entkopplung von physikalischer Hardware und Steuerungslogik kann jede Achse über einen simplen Index angesprochen werden. Dies ermöglicht es, Kernfunktionen wie das Starten und Stoppen der Motoren oder das Homen über alle Treiber mittels einfacher Schleifenkonstrukte umzusetzen, was die Skalierbarkeit und Wartbarkeit des Systems massiv erhöht.

### 3.1.1 Richtungssignale (DIR Pins)

Um die Richtung der Motoren zu bestimmen benötigt jeder Schrittmotortreiber einen DIR (Direction) Pin der entweder auf 1 um im Uhrzeigersinn zu drehen oder auf 0 um gegen der Uhrzeiger Sinn zu drehen. Um dies für alle 6 Motoren komfortabel und skalierbar zu lösen wurden Arrays implementiert.

```
GPIO_TypeDef* DIR_PORTS[6] = {TMC1_DIR_GPIO_Port, TMC2_DIR_GPIO_Port,
  TMC3_DIR_GPIO_Port, TMC4_DIR_GPIO_Port, TMC5_DIR_GPIO_Port,
  TMC6_DIR_GPIO_Port};
uint16_t      DIR_PINS[6]  = {TMC1_DIR_Pin, TMC2_DIR_Pin,  TMC3_DIR_Pin
  , TMC4_DIR_Pin, TMC5_DIR_Pin, TMC6_DIR_Pin};
```

Programmcode 3.1: Arrays für DIR Pins/Ports

### 3.1.2 Geschwindigkeitsregelung (STEP Pins)

Die Geschwindigkeitsregelung über der STEP Pins wird mittels Hardware Puls Weiten Modulation(PWM) Timern implementiert. Die Motortreiber brauchen zur Setzung der Geschwindigkeit eine Frequenz am STEP Eingang. Auch das ist durch Arrays realisiert um für Skalierbarkeit zu sorgen.

```
TIM_HandleTypeDef* AXIS_TIMERS[6] = {&htim2, &htim1, &htim3, &htim4, &
  htim8, &htim10};
uint32_t AXIS_CHANNELS[6] = {TIM_CHANNEL_1, TIM_CHANNEL_1,
  TIM_CHANNEL_1, TIM_CHANNEL_1, TIM_CHANNEL_1, TIM_CHANNEL_1};
```

Programmcode 3.2: Arrays für STEP Timer/Channels

### 3.1.3 Vergleich: Mit und ohne Array-Abstraktion

Ohne Array-Abstraktion würde die Motor-Ansteuerung mehrere if-else-Blöcke erfordern. Als Beispiel wird die Funktion zum Stoppen einer Motor-PWM von 3 Achsen gezeigt.

#### Anti-Pattern: Hard-codierte Hardware-Zuordnung

```
void Step_Stop_HardCoded(uint8_t ax) {
 if (ax == 0) {
  HAL_TIM_PWM_Stop(&htim2, TIM_CHANNEL_1);
  HAL_TIM_Base_Stop_IT(&htim2);
 }
 else if (ax == 1) {
  HAL_TIM_PWM_Stop(&htim1, TIM_CHANNEL_1);
  HAL_TIM_Base_Stop_IT(&htim1);
```

```
9   }
10  else if (ax == 2) {
11   HAL_TIM_PWM_Stop(&htim3, TIM_CHANNEL_1);
12   HAL_TIM_Base_Stop_IT(&htim3);
13  }
14
15 }
```

Programmcode 3.3: Motor-Stopp ohne Array-Abstraktion (Anti-Pattern)

Dieses Anti-Pattern führt zu mehreren Problemen:

- **Code-Duplikation**: Die gleiche Logik wird 6-mal wiederholt.
- **Wartbarkeit**: Änderungen müssen 6 mal vorgenommen werden.
- **Fehleranfälligkeit**: Leicht, einen Achsen-Fall zu vergessen oder falsch zu zuordnen.
- **Skalierbarkeit**: Das Hinzufügen/Entfernen einer Achse erfordert große Codeänderungen.

## 3.2 Initialisierung und SPI-Kommunikation der Motortreiber

### 3.2.1 SPI Protokoll

Das Seriel Peripheral Interface (SPI) ist ein Serielles Hochgeschwindigkeit Bussystem das vorwiegend zur Kommunikation von Mikrocontrollern mit Peripheriegeräten auf Platinen dient. SPI ist full duplex fähig, das heißt das bei jedem Takt 1 Bit vom Master an den Slave und zurück gesendet wird. Es besteht aus 3 Leitungen MOSI (Master Out $\longrightarrow$ Slave In), MISO (Master In $\longleftarrow$ Slave Out) und SCK (Serial Clock). Dazu kommt eine CS (Chip Select) Leitung die vom Master zu jeden Slave einzeln gehen muss. Nur wenn diese CS Leitung LOW ist sendet/empfängt der Slave Daten. Die Kommunikation erfolgt meist so das 2 Übertragungen nötig sind. In der ersten Übertragung sendet der Master ein Kommando an den Slave danach, da der Slave aber noch nicht bereit für eine Sinnvolle Rückmeldung ist sendet er nur Datenmüll. In der 2 Übertragung sendet der Master einen beliebigen Datensatz an den Slave, dieser hat inzwischen zeit gehabt seine Antwort auf das Kommando des Masters bereitzustellen

und sendet diese zum Master. Die Übertragungen sind jeweils synchron, heißt der Master und der Slave senden/empfangen synchron Daten.

#### SPI-Modis

Es gibt 4 SPI Mode 0-3. Diese Beschreiben sich wiefolgt:

| Mode | CPOL | CPHA |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 1 |
| 2 | 1 | 0 |
| 3 | 1 | 1 |

Abbildung 3.1: Diagramm der SPI Modi

CPOL (Clock Polarity):

- 0: Takt ist in Ruhe LOW, ein Wechsel auf HIGH zählt als steigende Taktflanke
- 1: Takt ist invertiert: in Ruhe HIGH, ein Wechsel auf LOW zählt als steigende Taktflanke

CPHA (Clock Phase):

- 0: Daten werden bei steigender Taktflanke (=abh. von CPOL) eingelesen, bei fallender ausgegeben
- 1: Daten werden bei fallender Taktflanke eingelesen, bei steigender ausgegeben

### 3.2.2 Initialisierung der Motortreiber

Die verwendeten Schrittmotortreiber vom Typ TMC5160 in Kombination mit NEMA-17 Schrittmotoren erfordern eine initiale Konfiguration ihrer internen Register, um

Parameter wie den Haltestrom, Run-Current und den Chopper-Modus festzulegen. Diese Kommunikation erfolgt über einen geteilten SPI Bus. Um die Treiber individuell adressieren zu können, wird vor jedem Datentransfer der spezifische Chip-Select (CS) Pin der Zielachse auf ein niedriges Potential (LOW) gezogen. Die TMC5160 erfordern den Betrieb in SPI Mode 3 (siehe 3.1)
Um Fehler direkt beim Initialisieren der Treiber zu erkennen, wurde eine Validierung eingebaut, ohne die der nächste Schritt nicht erreicht werden kann. Nach dem Schreiben der Konfigurationsdaten liest der Mikrocontroller das globale Statusregister (GSTAT) sowie aktuelle Treiberzustände zurück. Nur wenn das System valide Antworten (GSTAT = 0) detektiert, wird die Steuerung freigegeben. Störsignale oder fehlende Hardware-Verbindungen werden so unmittelbar beim Systemstart erkannt und führen zu einem sicheren Systemabbruch.
Die gewählte Konfiguration der Motortreiber ist wie folgt:

Tabelle 3.1: Konfiguration der TMC5160T Register via SPI

| **Register (Wert)** | **Funktion** |
|---|---|
| GSTAT (0x01) = 0x00000007 | Setzt das globale Statusregister zurück. Löscht Fehlerspeicher für Reset, Unterspannung der Charge-Pump und Driver-Shutdown vor dem Testlauf. |
| GCONF (0x00) = 0x00000004 | Aktiviert das Bit pdn_disable. Schaltet den Treiber in den internen Sensing-Modus um und erzwingt die Kommunikation rein über die SPI-Schnittstelle. |
| IHOLD_IRUN (0x10) = 0x00061008 | Konfiguriert den Motorstrom: Setzt den Fahrstrom (IRUN) auf ca. 50 %, den Haltestrom im Stillstand (IHOLD) auf ca. 25 % sowie eine Verzögerung (IHOLDDELAY) für die Stromabsenkung. |
| CHOPCONF (0x6C) = 0x14010053 | Konfiguration des Chopper-Verfahrens. Aktiviert den kraftvollen *SpreadCycle*-Modus, stellt die Schrittauflösung auf 16 Mikroschritte (mit automatischer Interpolation auf 256 Schritte) und definiert das Signal-Timing (*TOFF*, *TBL*). |
| PWMCONF (0x70) = 0xC10D0024 | Hinterlegt die Parameter für den flüsterleisen *StealthChop2*-Modus (Spannungsamplitude, Frequenz, Gradienten) für einen potenziellen, geschwindigkeitsabhängigen Moduswechsel. |

## 3.3 Finite State Machine (Endlicher Zustandsautomat)

Um die auf dem Mikrocontroller laufende Software übersichtlich zu gestalten und ein Selbstblockieren der Software zu verhindern, kommt eine Finite State Machine (FSM) zum Einsatz. Eine FSM ist ein mathematisches Modell für Systeme, die sich zu jedem

Zeitpunkt in genau einem von mehreren Zuständen befinden. Eine FSM besteht immer aus den folgenden Komponenten:

- Zustände (States): Die verschiedenen Phasen des Systems.
- Startzustand (Initial State): Der Zustand, in dem die FSM beginnt.
- Übergänge (Transitions): Die Bedingungen für den Wechsel von Zuständen.
- Ereignisse (Inputs): Die Ereignisse, die einen Wechsel bewirken.
- Aktionen (Outputs): Die Ausgaben der FSM.

### 3.3.1 Zustände

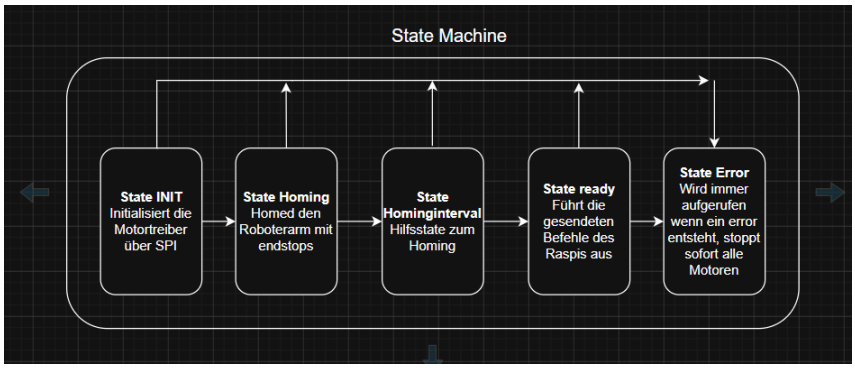



Abbildung 3.2: Diagramm der Finite State Machine

Für unseren Roboterarm wurden die Zustände `STATE_INIT`, `STATE_HOMING`, `STATE_HOMING_OFFSET`, `STATE_READY` und `STATE_ERROR` gewählt. Als Startzustand ist `STATE_INIT` definiert, da vor jedem Betrieb die Treiber initialisiert werden müssen. Die Funktionen der Zustände gestalten sich wie folgt:

## 3.4 Kinematische Referenzierungsfahrt (Homing)

Da Schrittmotoren über keine absolute Positionscodierung verfügen und aus Kostengründen auf absolute Encoder verzichtet wurde, ist der Systemzustand nach jedem Neustart undefiniert. Um den vordefinierten kinematischen Nullpunkt des Roboters zu ermitteln, muss zwingend eine Kalibrierungsfahrt (Homing) durchgeführt werden.

Tabelle 3.2: Funktionen der Zustände der Finite State Machine

| Zustand | Funktion |
|---|---|
| STATE_INIT | Initialisiert die Motortreiber über SPI, um die nötigen Parameter zu konfigurieren. |
| STATE_HOMING | Hier passiert der erste Teil der kinematischen Referenzfahrt, die den Roboter in die Home-Position bringt. Er fährt dabei den Endschalter der jeweiligen Achse an. |
| STATE_HOMING_OFFSET | Hier passiert der zweite Teil der kinematischen Referenzfahrt. Der Roboter fährt vom Endschalter in die vordefinierte Home-Position. |
| STATE_READY | Hier erfolgt der eigentliche Betrieb des Roboters. Der Mikrocontroller empfängt Befehle und führt diese aus, um den Roboter wie gewünscht zu bewegen. |
| STATE_ERROR | In diesen Zustand gelangt man nur im Fehlerfall. Es werden alle Motoren gestoppt und eine Fehlermeldung ausgegeben. |

### 3.4.1 Interrupt-Verarbeitung und Hardware-Entprellung

Für die Erfassung der Endschalter kommen externe Interrupts (EXTIs) zum Einsatz. EXTIs sparen erheblichen Rechenaufwand, da der Prozessor nicht permanent den Pin abfragen muss (Polling). Sobald eine Flanke erkannt wird, wird ein Interrupt-Request ausgelöst, der das Hauptprogramm pausiert und eine spezifische Interrupt-Service-Routine (ISR) wird aufgerufen. Diese Funktion setzt ein Flag, um dem Hauptprogramm zu signalisieren, dass der Endschalter getroffen wurde, das zu fahrende Offset zu berechnen ist und die Fahrtrichtung umgekehrt werden muss.

Um das mehrfache Auslösen des Interrupts zu verhindern, das durch sogenanntes Bouncen bei mechanischen Schaltern auftreten kann, ist in der Interrupt-Funktion eine Software-Entprellung eingebettet, die nach einem registrierten Interrupt alle weiteren Signale für 30 ms ignoriert.

Der Code dafür sieht wie folgt aus:

```
1 static uint32_t last = 0;
2 uint32_t now = HAL_GetTick();
3 if (now - last < 30) return;
4 last = now;
```

Programmcode 3.4: Debouncing für Interrupt-Verarbeitung

### 3.4.2 Berechnung kinematischer Offsets

Der mechanische Nullpunkt, an dem sich die Endschalter befinden, entspricht nicht dem kinematischen Nullpunkt des Roboters. Dies führt zu einem mathematischen Offset, das es auszugleichen gilt. Um dieses auszugleichen, muss die Winkeldifferenz in die entsprechende Schrittanzahl für die Motoren umgerechnet werden. Die Formel hierfür lautet:

$$\text{steps to move} = \frac{\Delta\text{rad}}{2 \cdot \pi} \cdot \mathit{SchritteproUmdrehung} \cdot \mathit{Mikroschritte} \cdot \mathit{Getrieberatio} \tag{3.1}$$

Mit den Konkreten eingesetzten Werten ergibt sich die Formel:

$$\text{steps to move} = \frac{\Delta\text{rad}}{2 \cdot \pi} \cdot 200 \cdot 16 \cdot 20 \tag{3.2}$$

- $\Delta$rad: Die Differenz des Offsets und der Home-Position in Radiant.
- 200: Die Vollschritte, die ein Schrittmotor pro Umdrehung besitzt.
- 16: Die eingestellte Mikroschritt-Auflösung pro Vollschritt.
- 20: Die mechanische Getriebeübersetzung (1:20) bei allen Achsen.

### 3.4.3 Sequenzieller Homing-Ablauf

Die Referenzierung des 6-Achs-Systems erfolgt sequenziell, dadurch werden mögliche Fehler wie eine Überschreitung des Mechanischen Limits durch mehrere gleichzeitig bis zum Anschlag fahrende Achsen grundsätzlich ausgeschlossen. Das Sequenzielle Homig hat auch den voreilt das es visuell auf korrektheit geprüft werden kann da man immer auf eine Achse nach der anderen Schauen kann. Der Ablauf erfolgt gesteuert durch eine globale Index-Variable (`current_homingaxis`), beginnend bei der Basis (Achse 0) bis zum Endeffektor (Achse 5). Der Ablauf für eine einzelne Achse gliedert sich dabei in folgende Phasen:
Zunächst verfährt die Achse im Zustand `STATE_HOMING` mit konstanter Geschwindigkeit auf den Endschalter zu. Nach Auslösen des Interrupts berechnet das Programm den erforderlichen Offset, invertiert das Richtungssignal (`DIR`) und wechselt in den Zustand `STATE_HOMING_OFFSET`. Erst wenn die exakt berechnete Schrittzahl für den Offset über den Timer-Interrupt abgefahren wurde, gilt die Achse als referenziert. Der Index wird

inkrementiert und die Prozedur für das nächste Gelenk gestartet, bis das System vollständig kalibriert ist und in den Betriebszustand `STATE_READY` übergeht.

## 3.5 Verbindung mir Raspberry Pi und normaler Betrieb

Im normalen Betrieb erhält der STM über UART Befehle von einem Raspberry pi 4b, der Raspberry sendet die Zielposition zu der sich der Endeffektor des Roboterarms bewegen soll in Radianten, also einen Array von 6 joint Winkeln und die Ausrichtung des Greifers. Außerdem noch eine Maximale Geschwindigkeit auf die die Motoren beschleunigen sollen

### 3.5.1 UART-Protokoll

# 4 High Level Software

Die High Level Logik Software auf dem Raspberry Pi dient um über eine GUI(Grafical User Interface) kartesische Koordinaten einzugeben zu dem ich der Endeffektor der Roboterarms bewegen soll. Diese x,y,z Koordinaten werden mithilfe von Inverser Kinematik mit der Python library IKpy zu 6 Jointwinkel in Radianten transformiert. Über die GUI kann man auch eine maximale Geschwindigkeit festlegen auf die die Motoren beschleunigen.

## 4.1 Inverse Kinematik und IKpy

### 4.1.1 Universal Robot Description File - URDF

## 4.2 Hypertext Markup Language-HTML

## 4.3 FastAPI

# 5 Hardware

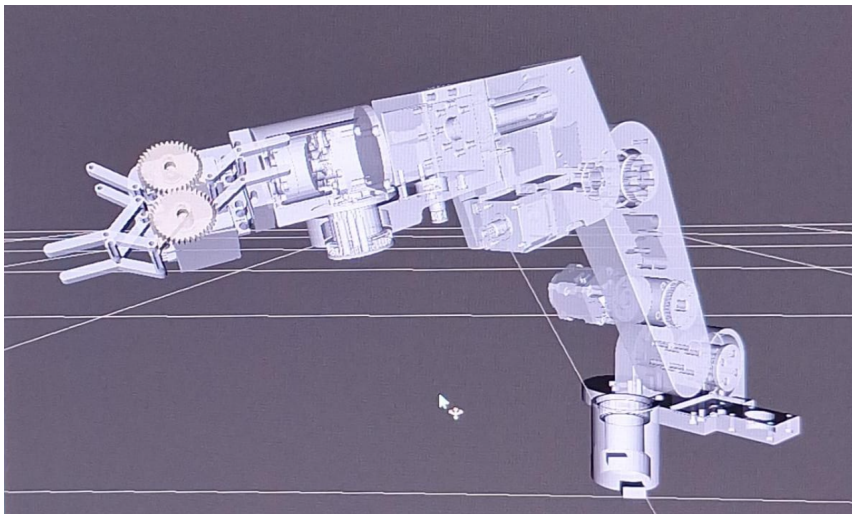


Abbildung 5.1: Blockdiagramm des 6-DOF-Roboterarms

# 6 Elektronikdesign und Hardware-Architektur

Dieses Kapitel beschreibt die Konzeptionierung, Komponentenauswahl und das Platinendesign (PCB) für die Steuerung des 6-DOF-Roboterarms. Die zu entwickelnde Platine fungiert als zentrales Management-System des Roboters. Sie nimmt die High-Level-Befehle des Raspberry Pi 4B entgegen, verarbeitet diese auf einem STM32F446RET6-Chip und steuert über eine dedizierte Leistungselektronik sechs TMC5160T Pro v1.0 Schrittmotortreiber an. Neben der Signalintegrität steht die stabile Spannungsversorgung der NEMA-17-Motoren im Vordergrund.

## 6.1 Systemarchitektur und Signalfluss

In diesem Abschnitt wird das grundlegende Blockschaltbild der Elektronik beschrieben. Das System teilt sich in drei logische Ebenen auf: High-Level-Recheneinheit (Raspberry Pi), Low-Level-Steuerung (STM32) und die Leistungsebene (TMC5160T Treiber). Die Kommunikation erfolgt über UART (Pi zu STM32) und einen gemeinsamen SPI-Bus für die Konfiguration der Treiberstufen.

Abbildung 6.1: Blockschaltbild der Elektronik-Architektur (Raspberry Pi → STM32 → Treiber → Motoren)

## 6.2 Komponentenauswahl und Begründung

### 6.2.1 Zentraler Mikrocontroller (STM32F446RET6)

Die Wahl fiel auf den STM32F446RET6. Ausschlaggebend hierfür waren die hohe Taktfrequenz von 180 MHz sowie das Vorhandensein von ausreichend Hardware-Timern, um sechs Schrittmotoren unabhängig und synchron per Hardware-PWM anzusteuern (siehe 3.1.2). Zudem bietet der Chip genügend GPIOs für die sechs separaten Chip-Select-Leitungen des SPI-Busses.

### 6.2.2 Schrittmotortreiber (TMC5160T Pro v1.0)

Für die Ansteuerung der NEMA-17-Schrittmotoren werden TMC5160T Pro v1.0 Module genutzt. Diese Treiber bieten neben einer hohen Stromtragfähigkeit vor allem die Anbindung über einen geteilten SPI-Bus, was eine dynamische Konfiguration der Motorströme erlaubt.

## 6.3 Mikrocontroller-Infrastruktur und Peripherie

Für den stabilen Betrieb des STM32F446RET6 wurde eine dedizierte Infrastrukturbeschaltung auf dem Board implementiert. Diese sichert die Signalqualität, die Programmierbarkeit und optionale Diagnosemöglichkeiten.

### 6.3.1 Taktversorgung (HSE) und Reset-Schaltung

Um die harten Echtzeitanforderungen bei der synchronen Generierung der Schrittfrequenzen zu erfüllen, wird der interne RC-Oszillator des STM32 durch einen präzisen, externen Quarzoszillator (High-Speed External, HSE) ersetzt. Dieser ist mit zwei Lastkondensatoren (C10, C11) und einem Serienwiderstand (R3) beschaltet. Die manuelle Systemrücksetzung wird über einen physischen Taster realisiert, der über ein RC-Glied hardwareseitig entprellt ist, um transiente Störungen auf der NRST-Leitung zu blockieren.

### 6.3.2 Programmierschnittstelle (SWD) und USB-C-Interface

Das Flashen und Debuggen der Firmware erfolgt über ein standardisiertes Serial-Wire-Debug-Interface (SWD) via Stecker J2 (SWDIO, SWCLK). Ergänzend wurde ein USB-C-Anschluss (J1) als USB-2.0-Full-Speed-Schnittstelle integriert. Um eine korrekte Erkennung an modernen USB-C-Host-Anschlüssen (z. B. PCs) zu gewährleisten, sind die Configuration-Channel-Leitungen (CC1, CC2) mit zwei Pull-Down-Widerständen gegen Masse gezogen. Ein Pull-Up-Widerstand an der D+-Leitung sorgt für die korrekte USB-Enumeration.

## 6.4 Schnittstellen und Bus-Topologie

### 6.4.1 UART-Interface zum Raspberry Pi

Die Kommunikation mit dem Raspberry Pi erfolgt über eine direkt angebundene UART-Schnittstelle. Hier werden die berechneten Ziel-Joint-Winkel in Radiant sowie die maximale Beschleunigung vom High-Level-System an den STM32 übertragen.

### 6.4.2 SPI-Bus und Chip-Select-Logik

Da sich alle sechs TMC5160-Treiber denselben SPI-Bus teilen (MOSI, MISO, SCK), muss jede Achse über eine eigene Chip-Select-Leitung (`CS`) exklusiv angesprochen werden.

### 6.4.3 STEP/DIR-Leitungen und Signalintegrität

Neben SPI benötigt jeder Treiber eine dedizierte `STEP`- und `DIR`-Leitung zur Vorgabe von Geschwindigkeit und Drehrichtung. Da diese Signale im harten Schaltbetrieb (MHz-Bereich) arbeiten, werden im Layout optionale Vorwiderstände zur Dämpfung von Signalüberschwingen eingeplant.

## 6.5 Spannungsversorgung und Power-Management

Aufgrund der hohen Eingangsspannung von 24 V und den unterschiedlichen Lastanforderungen der Systemkomponenten ist der Einsatz hocheffizienter DC/DC-Abwärtswandler (Buck Converter) zwingend erforderlich. Ein linearer Spannungsregler würde bei dieser Spannungsdifferenz eine unzulässige Verlustleistung $P_{\text{loss}} = (V_{\text{in}} - V_{\text{out}}) \cdot I$ in Form von Wärme erzeugen. Das System nutzt eine kaskadierte und parallele Schaltregler-Topologie, um vier getrennte Spannungsebenen bereitzustellen:

- **24 V Hauptversorgung:** Direkte Speisung der TMC5160-Leistungsstufen über den Terminal-Block `J3`.
- **12 V Lüfter-Versorgung:** Erzeugt aus den 24 V über einen synchronen Abwärtswandler. Diese Schiene dient exklusiv der Versorgung von aktiven Systemlüftern, um die Abwärme der Motortreiber effektiv abzuführen.

- **5 V Logik- und Pi-Versorgung:** Ebenfalls über einen Schaltregler U2 realisiert. Diese Schiene versorgt den Raspberry Pi 4B sowie die Peripheriesensorik. Die Stabilisierung erfolgt über ein LC-Ausgangsfilter.
- **3,3 V Mikrocontroller-Versorgung:** Abgestuft aus der stabilen 5 V-Schiene mittels eines kompakten Synchron-Abwärtswandlers. Dieser versorgt den STM32-Kern sowie die Digitalstufen der TMC5160-Treiber. Zur Entkopplung hochfrequenter Störungen der Digitalschaltungen ist die analoge Versorgungsschiene (`VDDA`) über eine Ferritperle (*Ferrite Bead*, `FB1`) und ein zusätzliches Filter-Netzwerk (`C6`, `C7`, `C8`) realisiert.

## 6.6 Prototyping und Hardware-Verifikation

Um die theoretische Konzeptionierung sowie die Low-Level-Softwarearchitektur vor dem finalen, integrierten Platinendesign zu validieren, wurde eine modulare Evaluierungsplatine entwickelt. Als Steuerungseinheit dient in dieser Phase ein externes *STM32-Nucleo*-Entwicklungsboard.

### 6.6.1 Architektur der Testplatine

Die Testplatine wurde als Single-Layer-Board realisiert. Sie dient als Bus- und Verteilerplatine für die sechs TMC5160T-Modulsteckplätze (`DRV_1` bis `DRV_6`). Die Signale für `STEP`, `DIR` und `CS` werden über Stiftleisten flexibel an das Nucleo-Board übergeben. Der SPI-Bus ist als gemeinsame Leiterbahnstruktur über alle Treiber hinweg geroutet.

### 6.6.2 Maßnahmen zur Signalintegrität und EMV

Da Schrittmotortreiber steile Stromflanken erzeugen, wurden auf dem Testboard gezielte EMV-Schutzmaßnahmen umgesetzt:

- **Lokale Dekopplung:** Jeder Treiber verfügt über eine direkte Pufferung aus einem 100 $\mu$F Elektrolytkondensator (transiente Spitzenströme) und einem 100 nF Keramikkondensator (hochfrequente Störungen) unmittelbar am Pin `VM`.
- **Überspannungsschutz:** Am 24 V-Eingang (`J2`) ist eine Suppressordiode (TVS-Diode, `D1`) platziert, welche die Gegen-EMK der Motoren beim abrupten Bremsen absorbiert.

- **Massive Stromschienen:** Die Versorgungsleitung für die Motorspannung wurde auf der Oberseite als großflächige, niederohmige Kupferschiene ausgeführt, um Spannungsabfälle zu minimieren.

## 6.7 Schaltplan und PCB-Layout (Integriertes Design)

### 6.7.1 Leiterbahndimensionierung und Stromtragfähigkeit

Für das finale Platinendesign (gefertigt über JLCPCB) müssen die Leiterbahnbreiten der Hauptstromversorgung mathematisch begründet werden, um Überhitzungen zu vermeiden. Die Berechnung der minimalen Leiterbahnbreite erfolgt nach der gängigen Industrienorm **IPC-2221**.
Die grundlegende Formel für den maximal zulässigen Strom $I$ in Abhängigkeit von der Querschnittsfläche $A$ und der zulässigen Temperaturerhöhung $\Delta T$ lautet:

$$I = k \cdot \Delta T^b \cdot A^c \tag{6.1}$$

Für Außenlagen (Top-/Bottom-Layer) gelten nach IPC-2221 die Konstanten $k = 0{,}048$, $b = 0{,}44$ und $c = 0{,}725$. Umgestellt nach der benötigten Querschnittsfläche $A$ (in $\text{mils}^2$) ergibt sich:

$$A = \left( \frac{I}{k \cdot \Delta T^b} \right)^{\frac{1}{c}} \tag{6.2}$$

### 6.7.2 Layer-Stackup und EMV-Konzept

Die finale Platine wird als 2-Layer-Platine umgesetzt. Um kritische Digitalsignale vor EMV-Einkopplungen zu schützen, wird ein striktes Massekonzept (*Star Grounding*) verfolgt, bei dem sich Logik-Masse und Leistungs-Masse sternförmig am Hauptkondensator des Netzteileingangs treffen.

### 6.7.3 Thermisches Management

Das Layout sieht unter den thermisch beanspruchten Treibern großflächige *Thermal Vias* vor, um die Abwärme der TMC5160-Stufen effizient in die inneren Kupferlagen und an die Umgebung abzuführen.

# Abbildungsverzeichnis

3.1 Diagramm der SPI Modi . . . . . 6
3.2 Diagramm der Finite State Machine . . . . . 8

5.1 Blockdiagramm des 6-DOF-Roboterarms . . . . . 13

6.1 Blockschaltbild der Elektronik-Architektur (Raspberry Pi → STM32 → Treiber → Motoren) . . . . . 14

# Quellenverzeichnis

[1] *WS2812B Intelligent control LED integrated light source Features and Benefits*. Adresse: https://cdn-shop.adafruit.com/datasheets/WS2812B.pdf

[2] *WS2812 Ansteuerung – Mikrocontroller.net*, Mikrocontroller.net, 2025. besucht am 10. Feb. 2025. Adresse: https://www.mikrocontroller.net/articles/WS2812_Ansteuerung?